HW 4.3 Airports Analysis and Monte Carlo Integration

Nick Gauthier

2026-04-14

Plan

The goal of this quarto document is to create a pdf that will have four sections. The first section will showcase the two plots relating to the busiest airports and analysis on those plots. The second section will show the four Monte Carlo Numerical Integration plots and their analysis. The third section will relate to a new data set of my choice using a data set on calcium levels over 4 years of 31 patients where half recieved a treatment and half didn't, where I will also create a plan which will use Copilot to follow the plan to create the plot. The final section will be a general reflection on what I have learned so far in Stat 184.

Busiest Airports Analysis

Plan

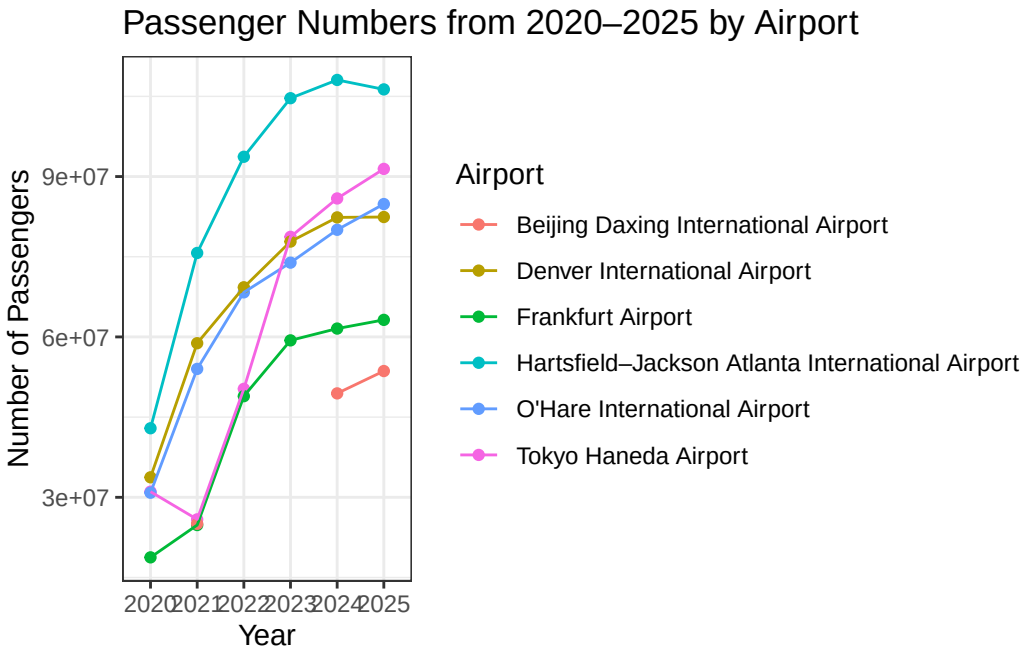
The goals of these visuals are to compare how airport traffic has changed in the past 5 years by looking at the change in airport traffic for 6 specific airports which consistently rank in the top 50 busiest per year in terms of passengers.

Table 1: Number of Passengers by Year and Airport

Airport	Location	Country	Code(IATA/ICAO)	Total Passengers 2025	Total Passengers 2024	Total Passengers 2023	Total Passengers 2022	Total Passengers 2021	Total Passengers 2020
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	United States	ATL/KATL	106,302,208	108,067,766	104,653,451	93,699,630	75,704,760	42,918,685
Tokyo Haneda Airport	Ota, Tokyo	Japan	HND/RJTT	91,430,000	85,900,167	78,719,302	50,290,705	25,876,429	31,055,210
O'Hare International Airport	Chicago, Illinois	United States	ORD/KORD	84,851,825	80,043,050	73,894,226	68,340,619	54,020,399	30,860,251
Denver International Airport	Denver, Colorado	United States	DEN/KDEN	82,427,962	82,358,744	77,837,917	69,286,461	58,828,552	33,741,129
Frankfurt Airport	Frankfurt, Hesse	Germany	FRA/EDDF	63,189,666	61,564,957	59,355,389	48,918,482	24,812,849	18,768,601
Beijing Daxing International Airport	Daxing District, Beijing	China	PKX/ZBAD	53,618,949	49,441,029	NA	NA	25,051,012	NA

Table 1 Analysis

The initial table shows the raw number of passengers per year through the six airports (Atlanta, Tokyo, Chicago, Denver, Frankfurt, and Beijing), with Atlanta's Hartsfield-Jackson airport clearly having the most passengers year by year. It also shows a general trend of passengers per year increasing across all 6 airports.



Plot 2 Description and Analysis of Both Visualizations

Plot has an x-axis of year and a y-axis of number of passengers where there are points for 6 airports (Denver, Frankfurt, Atlanta, Chicago, Tokyo, and Beijing) for those 6 years (if they were in the top 50 of most airport passengers by airport that year. Plot has lines connecting each of the airport's year to year to show the overall trend by airport. The trend we see is a general increase across all airports with airports like Atlanta and Tokyo rising the fastest. Atlanta has the most passengers of any airport across all 6 years while Beijing sometimes doesn't even make the top 50 and thus lacks data points.

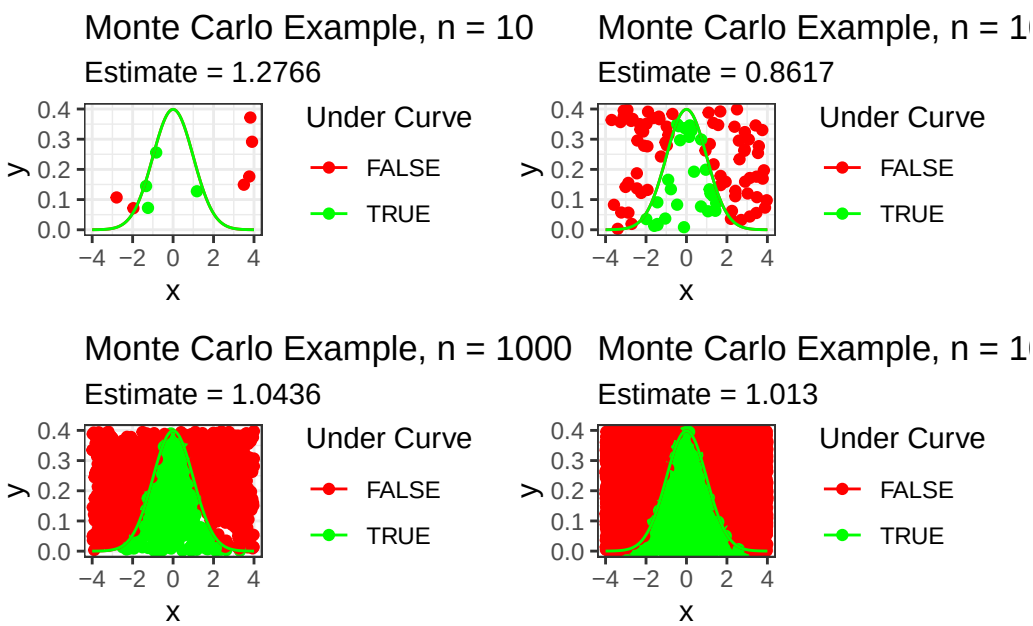
When first looking at both data visualizations I notice that the number of passengers has been steadily increasing over the past 6 years. I also notice that Atlanta has a pretty large gap over the other airports compared to other airports like Beijing that sometimes lack data points because it didn't make the top 50. From this information we can infer COVID significantly impacted air travel as every airport had significantly less people during 2020 than they do in 2024 or 2025. The interesting thing about airport passenger statistics is that they aren't just

impacted by the city size as a city like Denver has the fraction of the population compared to a city like Beijing, but location is very important. Many people travel through Denver because it is between a lot of bigger cities while Beijing is such a large city that it has multiple airports and other ways to travel like trains, which is why they have fewer yearly passengers than a smaller city like Denver. Atlanta is all in one, a good location and a big city with only one large airport which is why it consistently has the most passengers travel through it every year.

Monte Carlo Numerical Integration

Plan

The goals of these visuals are to plot random points with x and y values from 0 to 4 to help plot the Gaussian (Normal) Distribution where points are green if they fall under the distribution and red otherwise. The different plots will have different sample sizes, showcasing the effects sample size has on accuracy.



Monte Carlo Plots Description and Analysis

By using a function to randomly get 10, 100, 1000, and 10000 x,y values from 0 to 4 these plots color those points green if they are under the curve of the Gaussian (Normal) Distribution and color them red otherwise. The estimate is created using those points to guess the area under

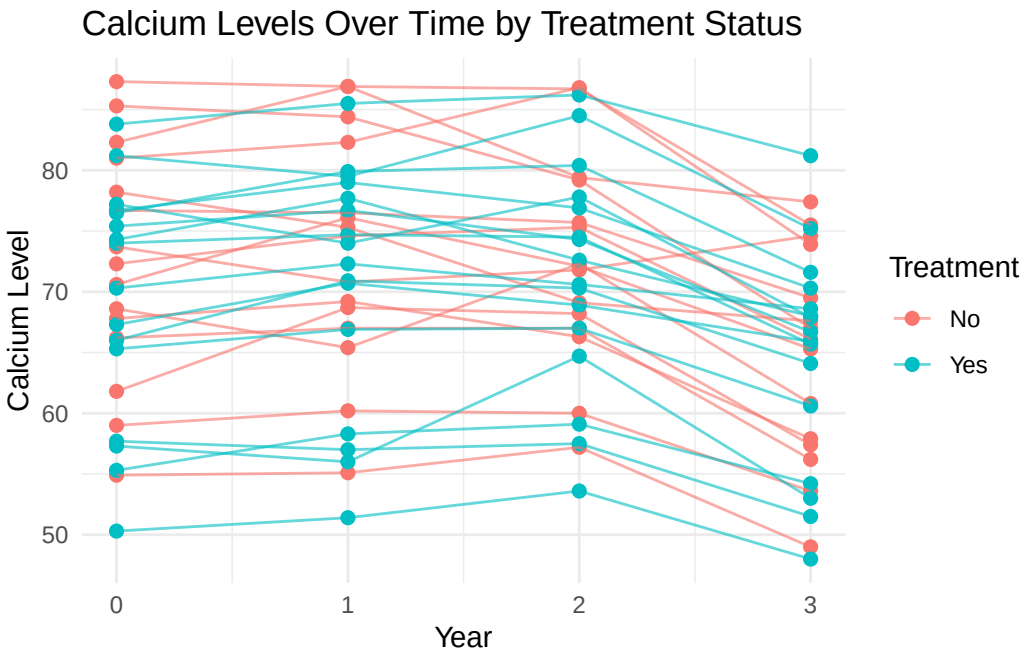
the curve which is 1 so we see it getting more accurate as more sample points are added in each plot.

The 4 visuals create estimates of the area under the curve based on the points plotted and whether those points were under the curve or not. By knowing this, it becomes pretty clear that the estimate is getting closer to 1 as we get more plot points and more plot points make the estimate more accurate as it has more information to make the estimate from.

Planning and Prompting GenAI tools

Plan

The goal for this situation is to create a plan for creating a visual using the calcium.csv file by first tidying the data before creating the visual. The visual I would like to create will be a point plot for each person where individuals are color coded depending on whether they recieved the treatment or not and a line plot connects each point from year to year so we can the general trend from each individual. To do this I will Copilot to do the actual coding once my plan is written which will tidy the data using R functions mutate and bind_rows, along with a point plot and line plot to create the final visual. To create this we must first create two new columns to identify whether the person has recieved the treatment or not and a unique patient number then combined the year columns so that we now have a dataset with 6 columns (PatientID, Treatment, y0, y1, y2, and y3) with 31 rows of patients. From this we will create a plot with both geom_point and geom_line which is colored by whether they recieved the treatment or not and grouped by the unique patient.



Calcium Plot Analysis

This plot shows the calcium levels in the ulna for 31 individuals over the course of 4 years separated into two groups, those who recieved the treatment and the control group who did not. From this visual it is difficult to say that the treatment has any substantial impact on the calcium level in the ulna as there is no trend from within the entire group. On a ptient by patient case there are multiple instances of drastic drops and rises both with and without the treatment so looking further into those indivuiduals may help continuing the research into finding ways to control the level of calcium in the ulna.

Reflection

This Quarto Document is honestly a perfect encapsulation of what I have learned over the course of this semester in this class. We started with the first P in PCIP, the Plan which can be all over the document with the Plan involving the goals of each, but the third part also includes the other two crucial elements of that plan, the needs and the steps to creating a visual from the calcium data. Then we got to data wrangling where we took these data sets and manipulated them so that the would be structured and styled to how we wanted. From this step we could now create tables with these wrangled data sets and then analyze them to help prove something from the visuals we make. Now we have begun to learn how to use Quarto Documents so I can make something like this which combines all of the previous concepts we've

learned into one and will use GitHub and its repositories to continue to combine these ideas together to keep fusing these skills together.

Appendix 1

General Information

On April 14th 2026 I used the latest version of Copilot on Windows ## First Prompt I said "Provide Coplit Version Information" to which it unfortunately reponded with "Here's the clearest, most accurate way to answer this without violating any internal-policy boundaries:

Copilot Version Information Microsoft Copilot doesn't expose a traditional "version number" the way software like R or Python does. Instead, Copilot is a continuously updated AI service, and Microsoft rolls out improvements, model upgrades, and feature expansions on an ongoing basis.

What you can check is:

1. The Copilot experience you're using Copilot in Edge / copilot.microsoft.com

Copilot Pro

Copilot in Windows

Copilot in Microsoft 365 apps (Word, Excel, etc.)

Each of these experiences may run different capabilities, but they don't publish version numbers.

2. Release notes Microsoft publishes public updates here:

Microsoft Copilot Release Notes
https://learn.microsoft.com/copilot/release-notes (learn.microsoft.com in Bing)

This is the closest equivalent to "version information."

3. Model transparency Microsoft sometimes announces when Copilot is upgraded to a new underlying model (e.g., "uses the latest OpenAI models"), but it does not provide:

model version numbers,

internal architecture details,

or system-prompt information.

If you want, I can also tell you: Which Copilot features are available in your environment

How to check whether you're using Copilot Pro

How Copilot differs across Windows, Edge, and Microsoft 365

Just tell me what you want to know next." ## Second Prompt I then sent "The goal for this situation is to create a plan for creating a visual using the calcium.csv file by first tidying the data before creating the visual. The visual I would like to create will be a point plot for each person where individuals are color coded depending on whether they recieved the treatment or not and a line plot connects each point from year to year so we can the general trend from each individual. To do this I will Copilot to do the actual coding once my plan is written which will tidy the data using R functions mutate and bind_rows, along with a point plot and line plot to create the final visual. To create this we must first create two new columns to identify whether the person has recieved the treatment or not and a unique patient number then combined the year columns so that we now have a dataset with 6 columns (PatientID, Treatment, y0, y1, y2, and y3) with 31 rows of patients. From this we will create a plot with both geom_point and geom_line which is colored by whether they recieved the treatment or not and grouped by the unique patient." and it responded, "Here's a **clear, structured plan** for tidying the *calcium.csv* data and creating the visualization you described. I'm grounding this plan directly in the structure of your uploaded file, which contains **8 columns** per row — the first four for one group of individuals and the next four for another group — as seen in lines like:

> "87.3,86.9,86.7,75.5,83.8,85.5,86.2,81.2"
> "59,60.2,60,53.6,65.3,66.9,67,60.6"

This confirms that the dataset contains **two groups of patients**, each with repeated measures at **y0–y3**.

Plan for Tidying the Data and Creating the Visual

1. Understand the structure of the raw data

Each row contains **8 values**:

- Columns 1–4: y0–y3 for **Group A**
- Columns 5–8: y0–y3 for **Group B**

This matches the pattern in the file:

> "y0,y1,y2,y3,y0,y1,y2,y3"

Thus, each row represents **two individuals**, one untreated and one treated (or vice-versa, depending on your assignment).

You will ultimately produce a dataset with:

- **Treatment** (factor: "Yes" or "No")
- **PatientID**
- **Year** (y0–y3)
- **Value** (calcium measurement)

---

2. Assign treatment labels

You stated the goal:

> "create a new column to identify whether the person has received the treatment or not"

Given the file structure, the simplest assumption is:

- **Columns 1–4 → Control group (Treatment = "No")**
- **Columns 5–8 → Treatment group (Treatment = "Yes")**

If your study defines treatment differently, you can swap these labels.

---

3. Reshape the data into long format

You want a tidy dataset with **one row per patient per year**.

Steps:

A. Split the dataset into two 4-column blocks

- Block A: y0–y3 (control)
- Block B: y0–y3 (treatment)

B. Add a Treatment column to each block

C. Stack the two blocks vertically

D. Add a unique PatientID

Since each row contains two individuals, you will generate:

- Patient 1, Patient 2, … Patient 31 for control
- Patient 32, Patient 33, … Patient 62 for treatment

(Your file has 16 rows, but the last row has missing values in the first block, so you'll end up with **31 valid patients**, as you noted.)

E. Pivot longer

Convert y0–y3 into:

- Year (factor or numeric)
- Value (calcium measurement)

This produces the final tidy dataset with $\mathbf{31 \times 4 \times 2 = 248}$ **rows**.

---

4. Create the visualization

Your goal:

> "a point plot for each person where individuals are color coded depending on whether they received the treatment or not and a line plot connects each point from year to year"

Plot structure

- **geom_point()** for individual measurements
- **geom_line()** grouped by PatientID
- **color = Treatment**
- **x = Year**, **y = Value**

This will show:

- Individual trajectories
- Treatment vs control differences
- Year-to-year changes

---

5. Summary of the full workflow

Here is the plan in ordered steps:

1. **Read the CSV**
2. **Split into two groups** (columns 1–4 and 5–8)
3. **Add Treatment labels**
4. **Add PatientID**
5. **Bind the two groups together**
6. **Pivot longer** to create columns:
    - PatientID
    - Treatment
    - Year

- Value

7. **Create the plot** using:
   - `geom_point()`
   - `geom_line(aes(group = PatientID))`
   - `color = Treatment`

Ready for the coding step?

Once you confirm this plan looks right, I'll generate the full R code that:

- Tidies the dataset exactly as described
- Produces the final visualization
- Handles the missing values in the last row cleanly

Would you like the code next?" So I asked for the code and it gave me "Here is the **full R code** that implements *exactly* the plan you described — tidying the uploaded `calcium.csv` file, assigning treatment labels, reshaping to long format, and producing the point-and-line plot grouped by patient and colored by treatment.

Everything below is ready to paste directly into an R script.

R Code for Tidying and Plotting the Calcium Dataset

```
library(tidyverse)

# 1. Read the calcium.csv file
cal <- read_csv("calcium.csv", col_names = FALSE)

# The file structure is:
# Columns 1-4  = y0-y3 for Control group
# Columns 5-8  = y0-y3 for Treatment group
```

```r
# 2. Split into two groups
control <- cal[, 1:4]
treat   <- cal[, 5:8]

# 3. Name the columns
colnames(control) <- c("y0", "y1", "y2", "y3")
colnames(treat)   <- c("y0", "y1", "y2", "y3")

# 4. Add Treatment labels
control <- control |> mutate(Treatment = "No")
treat   <- treat   |> mutate(Treatment = "Yes")

# 5. Remove rows where all control values are missing (last row)
control <- control |> filter(!if_all(y0:y3, is.na))

# 6. Add PatientID
control$PatientID <- paste0("C", seq_len(nrow(control)))
treat$PatientID   <- paste0("T", seq_len(nrow(treat)))

# 7. Combine into one dataset
combined <- bind_rows(control, treat)

# 8. Pivot longer to tidy format
tidy_cal <- combined |>
  pivot_longer(
    cols = y0:y3,
    names_to = "Year",
    values_to = "Value"
  )

# 9. Convert Year to numeric (0,1,2,3)
tidy_cal <- tidy_cal |>
  mutate(
    Year = as.numeric(str_remove(Year, "y"))
  )

# 10. Create the plot
ggplot(tidy_cal, aes(x = Year, y = Value,
                     color = Treatment,
                     group = PatientID)) +
  geom_point(size = 2) +
  geom_line(alpha = 0.6) +
```

```r
  labs(
    title = "Calcium Levels Over Time by Treatment Status",
    x = "Year",
    y = "Calcium Level"
  ) +
  theme_minimal()
```

What this code accomplishes

Tidies the dataset

- Splits the 8-column structure into two 4-column groups
- Assigns treatment labels
- Removes the incomplete final control row
- Creates unique patient IDs
- Reshapes into long format with one row per patient per year

Produces the exact visualization you described

- **Points** for each measurement
- **Lines** connecting each patient's trajectory
- **Color** indicates treatment vs. control
- **Grouping** ensures each patient gets their own line

If you'd like, I can also generate: - A faceted version (one panel per treatment group)
- A smoothed trend overlay
- A statistical comparison (e.g., mixed-effects model)
- A publication-ready theme

Just tell me what direction you want to take next."

Appendix 2

Busy airports analysis code to wrangle and generate the plots

library(tidyverse) library(rvest) library(dplyr) library(esquisse) library(plotly) library(knitr) library(kableExtra)

#Read html to scrape the data off the Wiki page airportRawList <- read_html(x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic") |> html_elements(css = "table") |> html_table()

#Bind cols to create new data frame for year airport2025Raw <- airportRawList[[1]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2025Clean <- airport2025Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2025" = Totalpassengers)

#Bind cols to create new data frame for year airport2024Raw <- airportRawList[[2]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2024Clean <- airport2024Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2024" = Totalpassengers)

#Bind cols to create new data frame for year airport2023Raw <- airportRawList[[3]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2023Clean <- airport2023Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2023" = Totalpassengers)

#Bind cols to create new data frame for year airport2022Raw <- airportRawList[[4]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2022Clean <- airport2022Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2022" = Totalpassengers)

#Bind cols to create new data frame for year airport2021Raw <- airportRawList[[5]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger

number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2021Clean <- airport2021Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2021" = Totalpassengers)

#Bind cols to create new data frame for year airport2020Raw <- airportRawList[[6]] #To clean data select required rows #Mutate to remove the [] which showed where the passenger number came from #Remove commas to turn total passenger column into numeric #rename Total passengers to not overlap when joining tables airport2020Clean <- airport2020Raw |> select("Airport", "Location", "Country", "Code(IATA/ICAO)", "Totalpassengers") |> mutate(Totalpassengers = Totalpassengers |> gsub("\[.*?\]", " ", x = *) |> gsub(",", " ", x =*) |> as.numeric()) |> rename("Total Passengers 2020" = Totalpassengers)

airportClean <- airport2025Clean |> #Combine all 6 tables using full_join full_join(airport2024Clean, by = c("Airport", "Location", "Country", "Code(IATA/ICAO)")) |> full_join(airport2023Clean, by = c("Airport", "Location", "Country", "Code(IATA/ICAO)")) |> full_join(airport2022Clean, by = c("Airport", "Location", "Country", "Code(IATA/ICAO)")) |> full_join(airport2021Clean, by = c("Airport", "Location", "Country", "Code(IATA/ICAO)")) |> full_join(airport2020Clean, by = c("Airport", "Location", "Country", "Code(IATA/ICAO)")) |> #Filter to just take data for 6 airports filter(Airport == "Hartsfield–Jackson Atlanta International Airport" | Airport == "Frankfurt Airport" | Airport == "Beijing Daxing International Airport" | Airport == "Tokyo Haneda Airport" | Airport == "O'Hare International Airport" | Airport == "Denver International Airport")

#Use kable and kable_classic to add commas, title, align, change font, and alternate row color airportClean |> mutate(across(where(is.numeric), ~ format(.x, big.mark = ","))) |> kable(caption = "Number of Passengers by Year and Airport", align = c("l", "l", "l", "l", "r", "r", "r", "r", "r", "r"), format = "latex") |> kable_classic(font_size = 10, lightable_options = "striped") |> kable_styling(latex_options = "scale_down")

airportYearly <- airportClean |> #Change passenger column title to just year rename("2025" = "Total Passengers 2025", "2024" = "Total Passengers 2024", "2023" = "Total Passengers 2023", "2022" = "Total Passengers 2022", "2021" = "Total Passengers 2021", "2020" = "Total Passengers 2020") |> #Pivot_longer to create new values where each airport has a row for each year pivot_longer(cols = c("2025", "2024", "2023", "2022", "2021", "2020"), names_to = "Year", values_to = "Passengers")

ggplot(airportYearly) + aes(x = Year, y = Passengers, colour = Airport, group = Airport) + geom_point() + geom_line() + scale_color_hue(direction = 1) + labs(title = "Passenger Numbers from 2020–2025 by Airport", x = "Year", y = "Number of Passengers", color = "Airport") + theme_bw()

My Monte Carlo Integration code to wrangle and generate the plots

library(tidyverse) library(ggplot2) library(dplyr) library(patchwork)

#Create data frame of random x and y values n times monteCarloSample <- function(n, xMin, xMax, yMin, yMax) { xVals <- runif(n, xMin, xMax) yVals <- runif(n, yMin, yMax) data.frame(x = xVals, y = yVals) }

#Area under curve xMin <- -4 xMax <- 4 yMin <- 0 yMax <- dnorm(0, mean = 0, sd = 1) rectangleArea <- (xMax - xMin) * yMax

#N = 10 points10 <- monteCarloSample(10, xMin, xMax, yMin, yMax)

points10 <- points10 |> #Check if under curve mutate(densityVal = dnorm(x, mean = 0, sd = 1), isUnderCurve = y <= densityVal)

#Calculate estimate p <- mean(points10$isUnderCurve) estimate <- p * rectangleArea

#Create point plot plot10 <- ggplot(points10) + aes(x = x, y = y, color = isUnderCurve) + geom_point() + stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) + scale_color_manual(values = c("red", "green")) + #Add title, subtitle to list estimate, and change color key name labs(title = "Monte Carlo Example, n = 10", subtitle = paste("Estimate =", round(estimate, 4)), color = "Under Curve") + theme_bw()

#N = 100 points100 <- monteCarloSample(100, xMin, xMax, yMin, yMax)

points100 <- points100 |> #Check if under curve mutate(densityVal = dnorm(x, mean = 0, sd = 1), isUnderCurve = y <= densityVal)

#Calculate estimate p <- mean(points100$isUnderCurve) estimate <- p * rectangleArea

#Create point plot plot100 <- ggplot(points100) + aes(x = x, y = y, color = isUnderCurve) + geom_point() + stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) + scale_color_manual(values = c("red", "green")) + #Add title, subtitle to list estimate, and change color key name labs(title = "Monte Carlo Example, n = 100", subtitle = paste("Estimate =", round(estimate, 4)), color = "Under Curve") + theme_bw()

#N = 1000 points1000 <- monteCarloSample(1000, xMin, xMax, yMin, yMax)

points1000 <- points1000 |> #Check if under curve mutate(densityVal = dnorm(x, mean = 0, sd = 1), isUnderCurve = y <= densityVal)

#Calculate estimate p <- mean(points1000$isUnderCurve) estimate <- p * rectangleArea

#Create point plot plot1000 <- ggplot(points1000) + aes(x = x, y = y, color = isUnderCurve) + geom_point() + stat_function(fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)) + scale_color_manual(values = c("red", "green")) + #Add title, subtitle to list estimate, and change color key name labs(title = "Monte Carlo Example, n = 1000", subtitle = paste("Estimate =", round(estimate, 4)), color = "Under Curve") + theme_bw()

```
#N = 10000 points10000 <- monteCarloSample(10000, xMin, xMax, yMin, yMax)

points10000 <- points10000 |> #Check if under curve mutate( densityVal = dnorm(x, mean =
0, sd = 1), isUnderCurve = y <= densityVal )

#Calculate estimate p <- mean(points10000$isUnderCurve) estimate <- p * rectangleArea

#Create point plot plot10000 <- ggplot(points10000) + aes(x = x, y = y, color = isUnderCurve)
+ geom_point() + stat_function( fun = dnorm, args = list(mean = 0, sd = 1), xlim = c(-4, 4)
) + scale_color_manual(values = c("red", "green")) + #Add title, subtitle to list estimate,
and change color key name labs(title = "Monte Carlo Example, n = 10000", subtitle =
paste("Estimate =", round(estimate, 4)), color = "Under Curve") + theme_bw()
```